Real-Time Translation with Azure AI Translator

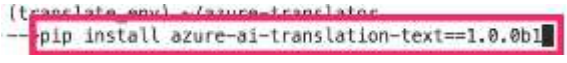
➔ Introduction:
- System and Software Requirements
  - Windows, MacOS, or Linux Machine
  - Microsoft Azure Portal
  - Python 3.8+, Java 17+
- Python Azure translation:
  - azure-ai-translation-text v1.0.0b1, azure-ai-translation-document v1.1.0b1
- Java Azure translation:
  - azure-ai-translation-text 1.0.0-beta.
- Python utilities:
  - requests v2.32.3, python-dotenv v1.0.1
- Java utilities: azure-core 1.50.0, dotenv-java 3.0.1
- Azure AI Translator
  - Cloud-based machine translation service built using modern neural machine translation technology. Can translate text and comments with a simple REST API call
  - Supports 100+ languages
  - Custom models for domain-specific terminology
  - Production-ready backend to integrate into apps
  - Built-in security - text input not logged during translation
- Azure AI Translator Features
  - Text Translation
  - Document Translation
  - Custom Translator

➔ Demo : Create Resource on Azure AI

➔ Installing Python important librariesin oreder to use Azure Ai language translator

➔ Install python python library)
- pip install python-dotenv

➔ command installs a specific beta version (1.0.0b1) of the Azure AI Translation Text package for Python



➔ writing code for translation by using python sdk – This is for single line

```python
import os

from azure.ai.translation.text import TextTranslationClient, TranslatorCredential
from azure.ai.translation.text.models import InputTextItem
from azure.core.exceptions import HttpResponseError
from dotenv import load_dotenv

load_dotenv()

key = os.getenv("TRANSLATOR_KEY")
endpoint = os.getenv("TRANSLATOR_ENDPOINT")
location = "eastus"

credential = TranslatorCredential(key, location)
text_translator = TextTranslationClient(endpoint=endpoint, credential=credential)

if __name__ == "__main__":
    text = "Thanks for always being there for me; I really appreciate our friendship."

    try:
        source_language = "en"
        target_languages = ["ko"]
        input_text_elements = [InputTextItem(text=text)]

        response = text_translator.translate(
            content=input_text_elements,
            to=target_languages,
            from_parameter=source_language
        )

        print(response)
    except HttpResponseError as exception:
```

- **Setup:** It imports necessary modules from azure.ai.translation.text and sets up authentication using a key and location (region).
- **Translation:** The TextTranslationClient translates a predefined English sentence into Korean (ko).
- **Execution:** The if __name__ == "__main__": block ensures the translation function runs when the script is executed.

➔ The response is generated in json format

➔ Code snnipet for language translation for a multiple input sentence

```python
def translate_sentences(sentences, target_language):
    try:
        input_texts = [InputTextItem(text=sentence) for sentence in sentences]
        response = text_translator.translate(
            content=input_texts, to=[target_language])

        translations = []

        for index, translation_item in enumerate(response):
            detected_language = translation_item.detected_language.language

            original_text = sentences[index]
            translated_text = translation_item.translations[0].text

            translations.append({
                "original": original_text,
                "translated": translated_text,
                "detected_language": detected_language
            })
        return translations
    except HttpResponseError as exception:
        print(f"Error Code: {exception.error.code}")
        print(f"Message: {exception.error.message}")
        return None
```

➔ `if __name__ == "__main__":`

- InputTextItem(text=sentence): This creates a new instance of an InputTextItem object for every sentence in your input list.
- for sentence in sentences: This iterates through each individual string in the provided sentences list.
- input_texts = [...]: The results are collected into a new list, which is then passed to the text_translator.translate method.

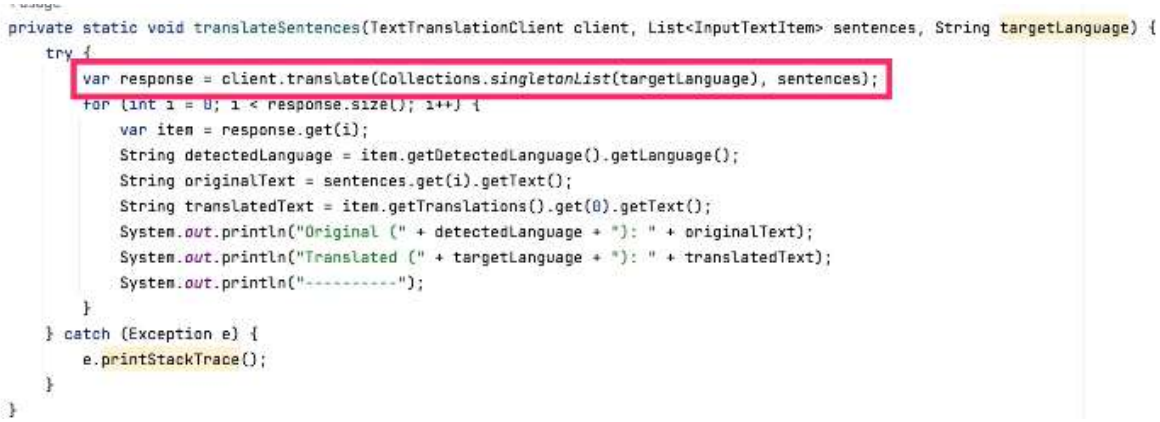
Demo : Translation by using The java SDK:

➔ Azure ai offer translation service in other language

➔ Reading multiple inputs

```java
public static void main(String[] args) {
    Dotenv dotenv = Dotenv.load();
    key = dotenv.get("TRANSLATOR_KEY");
    endpoint = dotenv.get("TRANSLATOR_ENDPOINT");
    location = "eastus";

    if (key == null || key.isEmpty()) {
        System.out.println("TRANSLATOR_KEY is missing in the .env file.");
        return;
    }
    if (endpoint == null || endpoint.isEmpty()) {
        System.out.println("TRANSLATOR_ENDPOINT is missing in the .env file.");
        return;
    }
```

➔ Translation



```
private static void translateSentences(TextTranslationClient client, List<InputTextItem> sentences, String targetLanguage) {
    try {
        var response = client.translate(Collections.singletonList(targetLanguage), sentences);
        for (int i = 0; i < response.size(); i++) {
            var item = response.get(i);
            String detectedLanguage = item.getDetectedLanguage().getLanguage();
            String originalText = sentences.get(i).getText();
            String translatedText = item.getTranslations().get(0).getText();
            System.out.println("Original (" + detectedLanguage + "): " + originalText);
            System.out.println("Translated (" + targetLanguage + "): " + translatedText);
            System.out.println("----------");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

- **Method Signature:** translateSentences takes a TextTranslationClient, a list of InputTextItem (the source text), and a targetLanguage string.
- **The Highlighted Line:**
- java
- var response = client.translate(Collections.singletonList(targetLanguage), sentences);
- Use code with caution.
- This sends a request to the Azure Translator API to translate the batch of sentences into the specified target language.
- **Processing Results:** The for loop iterates through the response items to:
- Identify the **detected source language**.
- Retrieve the **translated text**.
- Print both the original and translated versions to the console.
- **Key Details**
- **API Used:** Azure AI Translator (Text Translation).
- **Data Handling:** It uses Collections.singletonList(targetLanguage) because the API allows for multiple target languages in a single call, but this code only requests one.

Demo : Exploring Translation Options:

➔ Code snnipet for creaing api request to azure to check what language are supported.

```python
import os
import requests
import uuid
import json
from dotenv import load_dotenv

load_dotenv()

endpoint = os.getenv("TRANSLATOR_ENDPOINT")


def get_languages():
    path = '/languages'
    constructed_url = endpoint + path

    params = {
        'api-version': '3.0'
    }

    request = requests.get(constructed_url, params=params)
    response = request.json()

    return response

if __name__ == "__main__":
    languages = get_languages()
    print(json.dumps(languages['translation'], indent=4, ensure_ascii=False))
```

➔

- Environment Configuration: It uses dotenv to load configuration details, such as the API endpoint, from a .env file.
- API Request: The get_languages() function constructs a URL and sends a GET request to the /languages endpoint with API version 3.0 parameters.
- Response Handling: It parses the JSON response from the server and returns it.
- Main Execution: When run, it prints the retrieved list of languages specifically from the 'translation'

➔ Then authentication yourself

```
python list-of-languages.py
```

- **Command Purpose:** This command is used to execute a Python script named list-of-languages.py.
- **Context:** The script appears to be located within a directory structure related to a "translator" project, likely for listing supported languages for translation.
- **Execution:** The python command instructs the system to use the Python interpreter to run the code contained in the specified file

Demo:Bilingual Chart

➔ Bidirection custiomer support translation

➔ Create a code snnipet that takes end

```
def chat_interaction(customer_language, support_language):
    print("Start the chat interaction. Type 'exit' to end the session.\n")

    while True:
        customer_message = input("Customer: ").strip()
        if customer_message.lower() == 'exit':
            print("Ending chat session.")
            break

        translated_customer_message = translate_text(customer_message, support_language)
        print(f"Customer (Translated to Support): {translated_customer_message}")

        support_message = input("Support: ").strip()
        if support_message.lower() == 'exit':
            print("Ending chat session.")
            break

        translated_support_message = translate_text(support_message, customer_language)
        print(f"Support (Translated to Customer): {translated_support_message}")

if __name__ == "__main__":
    customer_language = input(
        """Enter the customer's language code (e.g., 'en' for English, 'fr'
```

- **body =**: This prepares the payload. The API expects a list of JSON objects, where each object contains the text you want to translate.
- **requests.post(...)**: This executes an HTTP POST request to the Azure endpoint. It includes:
- constructed_url: The API endpoint and path.
- params: Query parameters (like api-version and to language).
- headers: Authentication keys and content type.
- json=body: The text to be translated, formatted as JSON.
- **.json()**: This converts the raw API response into a Python dictionary/list.
- **return response[0]['translations'][0]**: This drills down into the nested JSON response to extract and return only the final translated string.
- ---
- **Key Requirements**
- To make this code functional, you must have the following variables defined elsewhere in your script:
- endpoint: Your Azure Translator resource URL.
- key: Your API subscription key.
- location: The Azure region (e.g., eastus).
- uuid: The uuid library must be imported (import uuid) to generate the unique trace ID.
-
- **body =**: This prepares the payload. The API expects a list of JSON objects, where each object contains the text you want to translate.

- **requests.post(...)**: This executes an HTTP POST request to the Azure endpoint. It includes:
- constructed_url: The API endpoint and path.
- params: Query parameters (like api-version and to language).
- headers: Authentication keys and content type.
- json=body: The text to be translated, formatted as JSON.
- **.json()**: This converts the raw API response into a Python dictionary/list.
- **return response[0]['translations'][0]**: This drills down into the nested JSON response to extract and return only the final translated string.
- ---
- **Key Requirements**
- To make this code functional, you must have the following variables defined elsewhere in your script:
- endpoint: Your Azure Translator resource URL.
- key: Your API subscription key.
- location: The Azure region (e.g., eastus).
- uuid: The uuid library must be imported (import uuid) to generate the unique trace ID.

Demo: ocalisation of Content

➔ How to make language translation offline

➔ Translates the code response and being bilingual get two languges

```python
import os
import requests
import json
from dotenv import load_dotenv

load_dotenv()

key = os.getenv("TRANSLATOR_KEY")
endpoint = os.getenv("TRANSLATOR_ENDPOINT")
location = "eastus"

def translate_text(text, target_language):
    path = '/translate'
    constructed_url = endpoint + path

    params = {
        'api-version': '3.0',
        'to': target_language
    }

    headers = {
        'Ocp-Apim-Subscription-Key': key,
        'Ocp-Apim-Subscription-Region': location,
        'Content-type': 'application/json'
    }
```

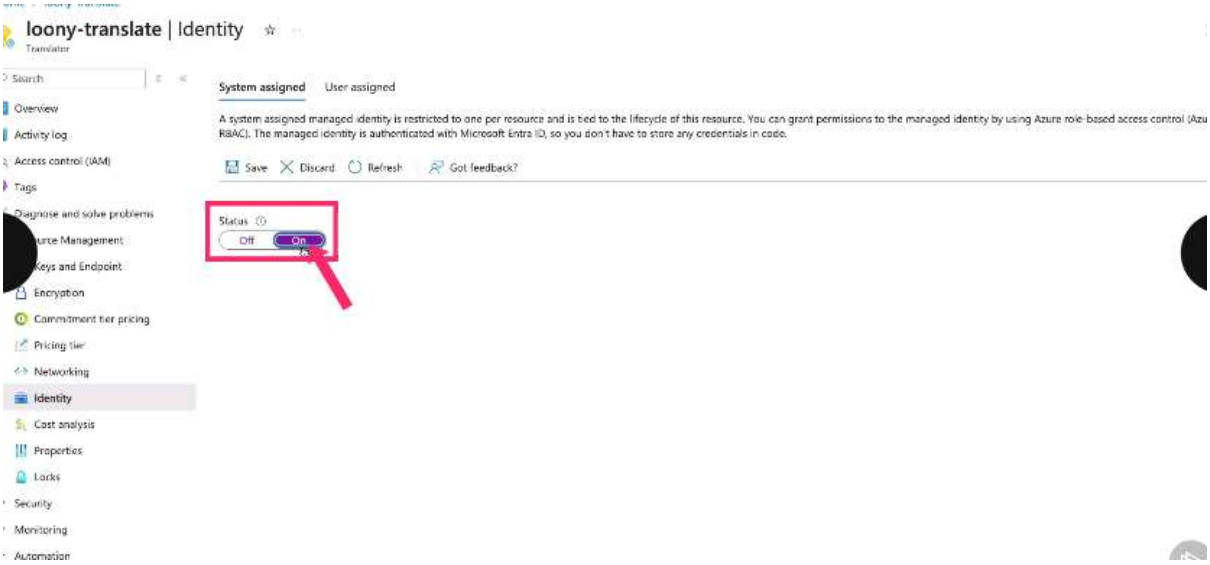
➔

```python
def localize_json(file_path, target_language):
    with open(file_path, 'r', encoding='utf-8') as file:
        data = json.load(file)

    def translate_object(obj):
        if isinstance(obj, dict):
            return {k: translate_object(v) for k, v in obj.items()}
        elif isinstance(obj, list):
            return [translate_object(i) for i in obj]
        elif isinstance(obj, str):
            return translate_text(obj, target_language)
        return obj

    localized_data = translate_object(data)

    output_file = f"localized_{target_language}.json"
    with open(output_file, 'w', encoding='utf-8') as file:
        json.dump(localized_data, file, ensure_ascii=False, indent=4)

    print(f"Localized JSON saved as {output_file}")

if __name__ == "__main__":
    target_language = input("Enter the target language code (e.g., 'en' for English, 'fr' for French, 'es' for Spanish): ").strip()
    localize_json('webpage_content.json', target_language)
```

➔

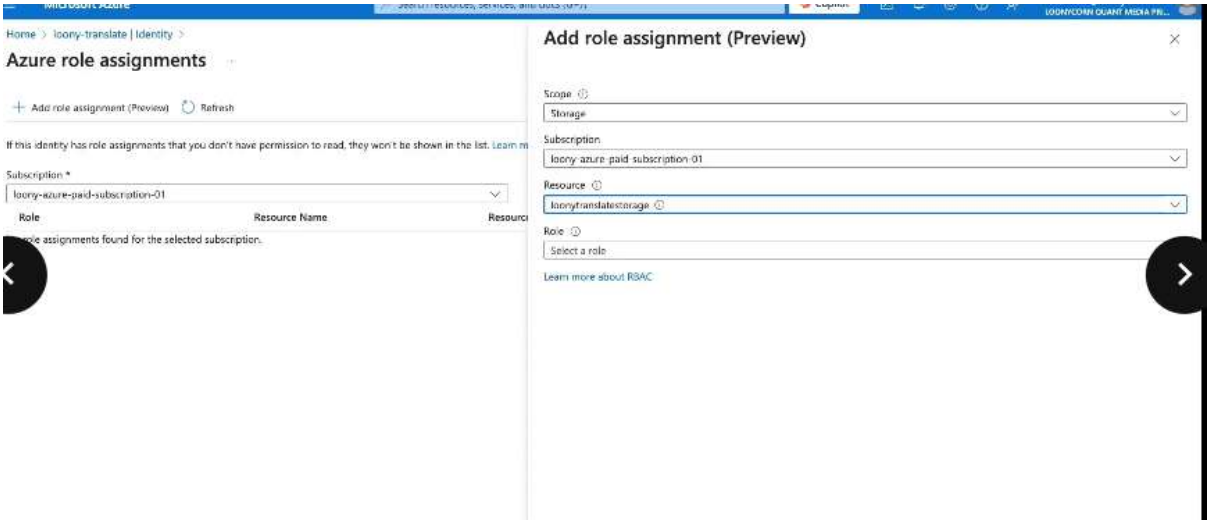
Demo: Using Azure Ai Translator

- ➔ This allows to preserve the azure ai translated format without changing
- ➔ String input in a differnt storage and the translated output in different container
- ➔ In source container you cn upload your input files
- ➔ Then enabling managed identities , managed identities lets you run with using an credentials
- ➔ Head over to identity on platform and turn it on and sav and assign all the necessary libraries for identities



➔

➔ Assign the permission by using azure role assignement and add role assignment



➔ Once this is created you will see tranlated resource